

Native Distribution Learning Guide

A research curriculum grounded in the vision_scan Flutter FFI plugin. Use this when you are ready to deepen how native code is built, packaged, published, and loaded on iOS, Android, and Windows.

Repo mental model: C/C++ (OpenCV + ZXing) -> platform binary (.xcframework / .so / .dll) -> GitHub Release zip -> consumer fetch (CocoaPods / SPM / Gradle / setup_windows) -> Dart FFI.

1. Big picture: what you already intuited

iOS: You ship an XCFramework. Inside it are one or more .framework slices (device, maybe simulator). Each framework contains compiled machine code plus headers/module maps. At app build time the linker embeds/links that code into the app. Dart then uses `DynamicLibrary.process()` because symbols are already in the process.

Android: You ship .so shared objects per ABI (arm64-v8a, armeabi-v7a, x86_64). They are packaged into the APK under `lib/<abi>/`. At runtime the dynamic linker loads them. Dart uses `DynamicLibrary.open('libvision_scan_native.so')`. Dependent .so files (OpenCV, libopencv_shared) must also be packaged.

Windows: You ship a .dll (plus OpenCV runtime DLLs). They must sit beside the Flutter runner .exe. Dart uses `DynamicLibrary.open('vision_scan_native.dll')`. Consumers run `dart run vision_scan:setup_windows` to fetch the zip.

2. Suggested learning order

- 1. C ABI, headers, extern "C", and export macros
- 2. Shared engine logic (native/src, qr_engine.h)
- 3. CMake fundamentals (SHARED libs, imported deps)
- 4. Android NDK cross-compile + jniLibs + Gradle packaging
- 5. iOS frameworks / XCFrameworks + xcodebuild
- 6. CocoaPods prepare_command vs SPM binaryTarget + checksum
- 7. Windows MSVC / Visual Studio CMake generator + DLL loading
- 8. Bash packaging scripts and deterministic zips
- 9. GitHub Actions: runners, tags, artifacts, releases
- 10. Dart FFI: DynamicLibrary, structs, memory ownership
- 11. Release engineering: version coupling and failure modes

3. Core topics to research (all platforms)

Machine code vs source distribution

Why it matters here: vision_scan does not ask consumers to compile OpenCV/ZXing. CI builds binaries; apps download them.

Research: *prebuilt binaries vs building from source; reproducible builds; binary compatibility*

Static linking vs dynamic linking

Why it matters here: ZXing is often built as .a (static) then linked into your shared product. OpenCV may remain a separate .so/.dll/.framework.

Research: *static vs shared libraries; rpath; DT_NEEDED; App Store bitcode/linking rules (historical)*

C ABI / calling conventions

Why it matters here: Dart FFI and C must agree on struct layout and symbol names (qr_engine.h).

Research: *ABI stability; packing/alignment; name mangling; extern "C"*

CMake

Why it matters here: All three scripts use CMake to produce SHARED libraries and wire OpenCV/ZXing.

Research: *add_library SHARED; IMPORTED targets; generators (Ninja, Visual Studio); toolchain files*

Cross-compilation

Why it matters here: Android builds on Linux/macOS for phone CPUs; iOS builds on macOS for arm64 device/sim.

Research: *host vs target; sysroot; Android NDK toolchain; Apple SDKs*

Versioning & release assets

Why it matters here: Tag v0.0.9 must match pubspec, podspec, android/build.gradle.kts, Package.swift URL.

Research: *semver; GitHub Releases; CDN caching; checksum integrity*

4. iOS-specific research list

Framework vs XCFramework

Why it matters here: scripts/build_ios.sh archives a framework then wraps with xcodebuild -create-xcframework.

Research: *Apple XCFramework docs; device vs simulator slices; Mac Catalyst pitfalls*

xcodebuild archive & destinations

Why it matters here: CI needs iOS platform installed; SUPPORTS_MACCATALYST=NO; generic/platform=iOS.

Research: *xcodebuild CLI; destinations; downloadPlatform; BUILD_LIBRARY_FOR_DISTRIBUTION*

lipo and fat binaries

Why it matters here: Simulator ZXing arm64 + x86_64 are merged before XCFramework creation.

Research: *lipo -create; thin vs fat; when XCFramework replaces fat frameworks*

CocoaPods vendored_frameworks + prepare_command

Why it matters here: ios/vision_scan.podspec downloads/symlinks vision_scan_native.xcframework at pod install.

Research: *CocoaPods podspec DSL; prepare_command; vendored binaries*

Swift Package Manager binaryTarget

Why it matters here: ios/vision_scan/Package.swift uses URL + SHA256 checksum of the exact zip.

Research: *SPM binary targets; swift package compute-checksum; Flutter plugin SPM layout*

DynamicLibrary.process() on iOS

Why it matters here: No open('lib...') - symbols come from the linked framework in the app process.

Research: *dyld; embed frameworks; Flutter ffiPlugin iOS linking*

Privacy manifests & system frameworks

Why it matters here: PrivacyInfo.xcprivacy + AVFoundation/CoreVideo/CoreMedia/UIKit linker settings.

Research: *Apple privacy manifests; Info.plist usage descriptions; camera entitlements*

5. Android-specific research list

ABIs and jniLibs layout

Why it matters here: Per-ABI folders under android/src/main/jniLibs or Gradle intermediates.

Research: *armeabi-v7a vs arm64-v8a vs x86_64; APK lib/ packing; split APKs / App Bundles*

Android NDK + CMake toolchain

Why it matters here: scripts/build_android.sh uses android.toolchain.cmake and ANDROID_NDK_HOME.

Research: *NDK r26+; ANDROID_ABI; ANDROID_PLATFORM; libc++_shared.so*

Shared object dependencies

Why it matters here: libvision_scan_native.so needs libopencv_java4.so (and often libc++_shared.so) beside it.

Research: *readelf/nm/ldd (or Android llvm-readelf); SONAME; load order*

Gradle prepareNativeLibs download pattern

Why it matters here: android/build.gradle.kts downloads android-<abi>.zip from GitHub Releases if local jniLibs missing.

Research: *Gradle tasks; sourceSets.jniLibs; caching; flutter clean behavior*

DynamicLibrary.open on Android

Why it matters here: open('libvision_scan_native.so') - name must match add_library(...) output.

Research: *System.loadLibrary; JNI_OnLoad (optional); Dart FFI without JNI*

6. Windows-specific research list

MSVC / Visual Studio CMake generator

Why it matters here: scripts/build_windows.sh uses -G "Visual Studio 17 2022" -A x64; runners must pin windows-2022 when VS2022 is required.

Research: *CMake generators; vswhere; msvc-dev-cmd; Ninja+MSVC alternative*

DLL search path

Why it matters here: vision_scan_native.dll + opencv_world*.dll copied next to the Flutter runner exe.

Research: *Windows DLL search order; delay-load; dependency walker / dumpbin*

Consumer setup CLI

Why it matters here: bin/setup_windows.dart downloads windows-x64.zip from releases/latest (or --url).

Research: *Dart CLI executables in pubspec; archive extraction; side-by-side deploy*

7. Scripting & packaging to research

Bash for build orchestration

Why it matters here: scripts/build_ios.sh, build_android.sh, build_windows.sh, update_ios_spm_checksum.sh.

Research: *set -euo pipefail; env vars; curl/unzip; path quoting; CI vs local*

Deterministic archives

Why it matters here: CI touches timestamps and uses zip -X so SPM checksums stay stable across retags.

Research: *reproducible builds; zip metadata; SOURCE_DATE_EPOCH*

Artifact verification

Why it matters here: file / lipo -info / swift package compute-checksum / shasum.

Research: *supply-chain integrity; checksums vs signatures*

8. GitHub Actions & CI to research

This repo's `.github/workflows/native.yml` is the living example. Topics:

- Triggers: push tags v*, workflow_dispatch
- Runners: macos-26 (iOS/Xcode), ubuntu-latest (Android/NDK), windows-2022 (MSVC)
- Caching vs downloading large deps (OpenCV zips from scanner-binaries)
- actions/checkout, setup-java, setup-android, ilammy/msvc-dev-cmd
- upload-artifact vs softprops/action-gh-release assets
- Permissions: contents: write for attaching release files
- Xcode platform installs: xcodebuild -downloadPlatform iOS
- Android SDK package drift: obsolete 'tools' package vs platform-tools
- Gates: SPM checksum validation after packaging
- Force-moving tags and why release engineering often needs two passes

What to study beyond this repo

Why it matters here: Same patterns appear in any binary-shipping Flutter/Rust/C++ plugin.

Research: *GitHub Actions docs*; *OIDC*; *reusable workflows*; *matrix builds*; *caching*

9. Dart FFI topics tied to this plugin

- `DynamicLibrary.process` vs `open` - see `lib/src/ffi/native_bindings.dart`
- Struct mirroring of C types (`VSFrameResult`, `VSFinalResult`, ...)
- Ownership: who mallocs / who `free_qr_string` / `free_qr_bytes`
- Isolates: calling native from a background isolate
- `ffiPlugin`: true in `pubspec.yaml` vs `MethodChannel` plugins
- Smoke tests: ping / `opencv` / `zxing` exported symbols

10. vision_scan file map (cheat sheet)

<code>native/include/qr_engine.h</code>	C API
<code>native/src/native.cpp</code>	shared engine (Android/iOS path)
<code>native/android/CMakeLists.txt</code>	Android .so build
<code>native/ios/vision_scan_native/</code>	Xcode framework project
<code>native/windows/CMakeLists.txt</code>	Windows DLL build
<code>scripts/build_{ios,android,windows}.sh</code>	
<code>ios/vision_scan.podspec</code>	CocoaPods download/vendor
<code>ios/vision_scan/Package.swift</code>	SPM binaryTarget
<code>android/build.gradle.kts</code>	download .so zips
<code>bin/setup_windows.dart</code>	Windows consumer setup
<code>.github/workflows/native.yml</code>	CI release pipeline
<code>lib/src/ffi/native_bindings.dart</code>	Dart FFI loader

11. Glossary

XCFramework: Apple multi-platform binary bundle (e.g. ios-arm64 + simulator).

Framework: Single-platform Apple package: binary + headers + modules.

ABI (Android): CPU slice folder: arm64-v8a, armeabi-v7a, x86_64.

.so: ELF shared object - Android dynamic library.

.dll: Windows dynamic library.

.a: Static archive linked at build time (e.g. ZXing).

NDK: Android Native Development Kit for cross-compiling C/C++.

CMake: Build system generator used on all three platforms.

CocoaPods: iOS dep manager; podspec prepare_command vendors XCFramework.

SPM: Swift Package Manager; binaryTarget needs URL + checksum.

Checksum: SHA-256 of the exact iOS zip SPM will download.

jniLibs: Android convention for packaging .so into the APK.

FFI: Dart calling C without MethodChannel for the scan path.

dyld / loader: OS dynamic linker that maps shared libraries at runtime.

GitHub Release: Versioned host for platform zip assets on tag v*.

12. Practice projects (when ready)

- Build only ZXing for one Android ABI and inspect the .a/.so with llvm-readelf.
- Create a tiny XCFramework from a hello.c and load it from a sample app.
- Write a Gradle task that downloads a zip and wires jniLibs like prepareNativeLibs.
- Add a GitHub Actions job that publishes a zip on tag and prints its checksum.
- Break SPM on purpose (wrong checksum) and read the Xcode error - then fix it.
- Compare DynamicLibrary.process vs open by logging loaded images (iOS) / maps (Android).

13. Recommended external reading (search terms)

- Apple: Distributing binary frameworks as Swift packages
- Apple: Creating an XCFramework
- Android: Distribute your app native libraries / NDK CMake guides
- Flutter: Bind to native code using FFI / legacy ffiPlugin
- Flutter: Swift Package Manager for plugin authors
- CMake: cmake-toolchains and imported targets
- GitHub Docs: Using workflow run artifacts and releasing binaries
- ELF/DLL: How dynamic loaders resolve dependencies

Generated for the vision_scan repository. Revisit this guide whenever you change how binaries are built or published - distribution knowledge is the difference between "it works on my machine" and "consumers can upgrade to v0.0.9 cleanly."